

Robot-Factored World Models via Robot Rendering

Byungjun Kim¹ Taeksoo Kim¹ Hyunsoo Cha¹ Hanbyul Joo^{1,2}

¹Seoul National University ²RLWRLD

{byungjun.kim, taeksu98, 243stephen, hbjo0}@snu.ac.kr

Edition note. Prepared with Codex for personal reading from [arXiv:2607.22535v1](#) (24 July 2026). Prose and captions have been edited for clarity; technical content, equations, numerical results, figure artwork, references, and section numbering are preserved. This is not an author-approved revision. “We” retains the original authors’ voice. Pagination differs from the source.

Abstract

Action-conditioned video world models predict future observations from an initial observation and an action signal. Robot actions affect future observations through two distinct processes. The robot body and controller first realize each action as robot motion. The scene then responds through contact and object motion. A world model conditioned directly on action commands must learn the action-realization process. Conditioning on logged future states leaks the interaction outcomes that the model should predict. We propose robot-factored world models that handle two robot-specific factors outside the world model. For action realization, we roll each command through the robot’s own controller and kinematics to produce a nominal trajectory. This intermediate signal is available at deployment and avoids both action-realization learning within the world model and future-state leakage. For robot rendering, we render the nominal trajectory using the robot URDF. These renderings represent the robot’s geometry, kinematics, and appearance explicitly, factoring those properties out of the world model. We pair end-effector depth with scene depth to resolve depth ambiguity. This pair provides geometric cues for contact and occlusion beyond image-plane overlap. We combine camera-aware static RGB/depth context and rendered robot geometry into a shared visual world-model interface that stays consistent across viewpoints and robot embodiments. The model sees the action only as visible robot geometry and learns how objects respond to it. Our experiments show that the rendered interface outperforms vector-conditioned baselines and generalizes to unseen robot embodiments at inference. We also demonstrate robot manipulation video generation from human demonstrations: we retarget the hand motion, render it as robot geometry, and pass it to the model.

Keywords: robot world models, action-conditioned video generation

1 Introduction

World models predict how an environment evolves in response to an agent’s actions. These predictions let us anticipate behavior without acting in the real world [1]. Recent advances in video generative models have enabled world models that predict future observations directly in pixel space [2, 3, 4, 5]. A video world model conditioned on a candidate robot action can roll out the future that action would produce. These rollouts can serve as a learned environment for policy improvement [6] or for ranking behaviors before real execution [7]. We study how to present robot actions to such a model.

A physical robot realizes actions through an embodiment-specific process before any scene interaction occurs. A world model conditioned directly on robot-specific control signals must therefore learn both

how the robot realizes those signals as motion and how the scene responds to that motion. Existing robot video world models largely keep both tasks inside the network. They inject robot control signals through conditioning layers [3, 4, 5], so the signal that the model receives remains tied to an embodiment-specific action representation. Visual Action Prompts (VAP) [8] suggest representing actions as image-space visual conditions. This approach requires us to choose which robot signal to render. VAP renders logged robot state trajectories, which align visually with the future video. These logged states already contain the realized outcomes of scene interaction: contact, compliance, latency, and closed-loop corrections. Their renderings therefore provide aligned prompts but leak the interaction that the world model should predict. We need a visual conditioning signal that is available at deployment and leaves scene-response prediction as the model’s task.

The nominal trajectory is the deployment-available conditioning signal in our robot-factored visual world-model interface. We produce this trajectory by rolling an action sequence through the robot’s own controller and kinematics. It describes the robot motion realized from the actions before we observe scene interaction. The nominal trajectory is available at inference, agrees with realized motion in free space, and diverges from realized motion where the model should predict contact-mediated interaction. We render the nominal trajectory using the robot URDF to produce camera-aligned robot geometry. Camera-aware static RGB/depth context and this rendered geometry form a shared visual world-model interface for fixed and dynamic view-points. The model sees the action as visible robot geometry and learns how the scene responds around it. The rendered geometry localizes the action in image space, but RGB alone cannot resolve the robot’s spatial relationship to the scene. We add end-effector and scene depth to provide geometric cues for proximity, contact, and occlusion beyond image-plane overlap.

The interface represents actions as rendered robot geometry, decoupling world-model conditioning from robot-specific control signals and action representations. This design supports generalization to unseen robot embodiments and grounded robot-interaction video generation from retargeted human demonstrations.

We make three contributions:

- (1) We introduce a deployment-realistic visual world-model interface. We realize actions as nominal robot trajectories and render them as camera-aligned URDF mesh RGB and end-effector depth. Camera-aware static RGB/depth context factors out scene appearance and viewpoint.
- (2) We make the interface depth-aware by adding end-effector and scene depth. These depth signals provide geometric cues for proximity, occlusion, and likely contact beyond RGB image-plane overlap.
- (3) We show that the same rendered interface supports different robot embodiments and motion sources. This shared interface enables generalization to unseen robot embodiments and grounded robot-interaction video generation from retargeted human demonstrations.

2 Related Work

Action-Conditioned Robot World Models. World models predict how observations evolve under actions. Researchers have used these models for imagination, planning, policy evaluation, and policy improvement [1, 9, 10, 11, 12]. Recent video world models extend this approach to pixel-space rollouts for games, driving, navigation, and robot manipulation [13, 14, 7, 6, 3, 4, 5, 15, 16, 17, 18, 19, 20, 21, 22, 2]. These approaches differ primarily in their action representations and the interfaces through which they present actions to the model. Some models condition directly on robot-specific action signals [3, 4, 7]. Others learn latent or abstract action representations from large-scale video [5, 23, 24, 25, 26, 27]. Direct action

interfaces remain tied to embodiment-specific control spaces. Latent-action approaches hide embodiment geometry behind learned abstractions. We condition our model on nominal trajectories realized before scene interaction. This conditioning exposes embodiment geometry and avoids future-interaction leakage.

Visual Conditioning for Video Generation. Researchers use visual conditioning as a standard mechanism to steer image and video diffusion models with spatial signals such as depth, pose, edges, masks, optical flow, trajectories, and reference frames [28, 29, 30, 31, 32, 33, 34, 35, 36, 37]. Other work uses object motion, camera motion, drag edits, and point tracks as direct video conditions [38, 39, 40, 41, 42, 43]. These works establish visual conditioning as a general mechanism for controlling image and video generation.

Visual controls for embodied video prediction can represent the acting body itself. Visual Action Prompts [8] render robot states as image-space prompts. Dexterous World Models [44] condition video generation on static scene renderings and hand mesh trajectories to model interaction-induced scene dynamics. We share the visual-conditioning principle of these approaches but render a different robot signal. We render the nominal trajectory computed before scene interaction, rather than logged states that already reflect interaction outcomes.

3 Method

3.1 Robot-Factored Visual World-Model Interface

An action-conditioned robot world model predicts a future video $\mathbf{V}_{1:F}$ from the current observation and a proposed action sequence $\mathbf{a}_{1:F}$. If we condition the video model directly on $\mathbf{a}_{1:F}$, it must learn two distinct processes: how the robot turns actions into motion, and how the scene responds to that motion. Action realization depends on the robot’s embodiment and controller. Scene-response prediction is the shared problem that the world model should solve. We therefore separate the two processes. We explicitly model action realization before supplying its result as conditioning to the world model.

We represent this realization process with an operator Φ_R that maps actions into a nominal trajectory:

$$\mathbf{q}_{1:F} = \Phi_R(\mathbf{a}_{1:F}; \mathbf{q}_0), \quad (1)$$

where $\mathbf{q}_{1:F}$ is the nominal robot trajectory produced by the action before scene interaction. We then render this nominal trajectory along the target camera trajectory $\mathcal{C}_{1:F}$:

$$(\mathbf{M}_{1:F}^{\text{rgb}}, \mathbf{D}_{1:F}^{\text{eff}}) = \Pi_R(\mathbf{q}_{1:F}; \mathcal{C}_{1:F}), \quad (2)$$

where $\mathbf{M}_{1:F}^{\text{rgb}}$ is the robot mesh RGB video and $\mathbf{D}_{1:F}^{\text{eff}}$ is end-effector-only depth. The rendering operator Π_R uses the robot URDF and geometry to project the nominal trajectory into the target camera frame. In parallel, we render the initial scene state $\mathbf{S}_0$ along the same camera trajectory to produce a static context stream:

$$(\mathbf{B}_{1:F}^{\text{rgb}}, \mathbf{D}_{1:F}^{\text{scene}}) = \Pi_S(\mathbf{S}_0; \mathcal{C}_{1:F}), \quad (3)$$

where $\mathbf{B}_{1:F}^{\text{rgb}}$ and $\mathbf{D}_{1:F}^{\text{scene}}$ provide appearance and geometric context in the target camera coordinates. We use these streams to condition the world model, which learns

$$p_\theta \left(\mathbf{V}_{1:F} \mid \mathbf{B}_{1:F}^{\text{rgb}}, \mathbf{D}_{1:F}^{\text{scene}}, \mathbf{M}_{1:F}^{\text{rgb}}, \mathbf{D}_{1:F}^{\text{eff}}, \mathcal{T} \right), \quad (4)$$

where $\mathcal{T}$ is the text prompt supplied to the video model. The prompt contains scene context only; it excludes descriptions of the intended action or future outcome.

We thus convert robot-specific control signals into visible robot geometry that the model can use. We keep the realization and rendering operators fixed as preprocessing steps. We leave the learned model with the shared task of predicting the scene’s response to the rendered robot motion.

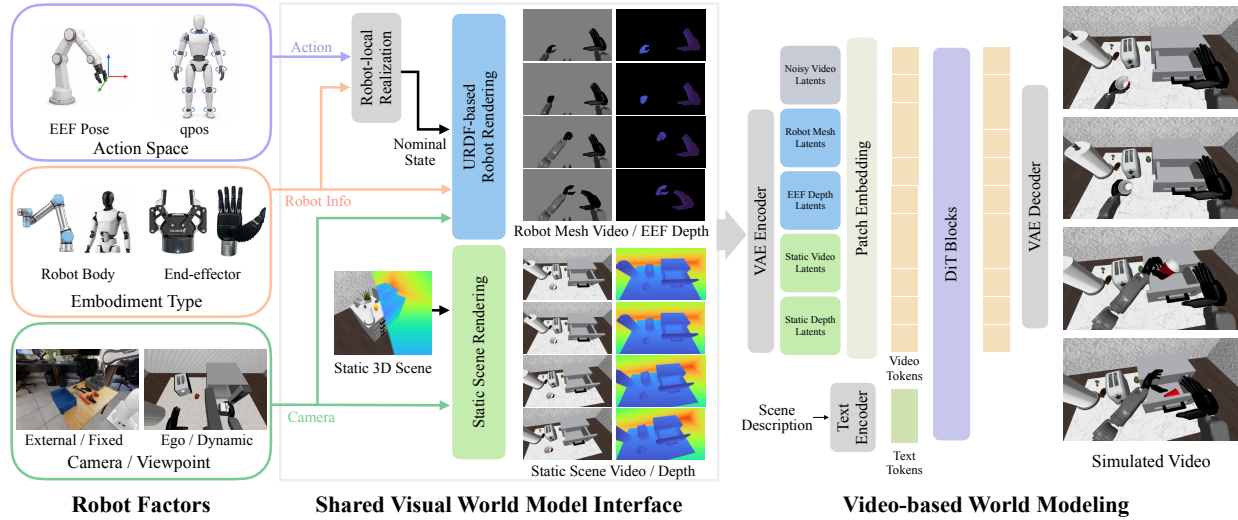


Figure 1. Visual world-model interface. Static context supplies scene and viewpoint information. Rendered nominal robot geometry represents the action. The diffusion model uses these inputs to predict the scene’s response.

3.2 Nominal Trajectory Conditioning

A visual conditioning signal must meet two requirements for deployment: we must be able to obtain it at inference time, and it must convey information about the robot motion produced by the action.

The *raw action* $\mathbf{a}_{1:F}$ consists of commands from a policy, teleoperator, or dataset controller, such as joint-space or task-space control commands. These commands are available at deployment, but their meaning depends on the particular robot and controller.

The *nominal trajectory* $\mathbf{q}_{1:F}$ is the robot-only motion produced by the robot’s own controller and kinematics before we observe scene interaction. It lies between action and interaction: we can obtain it at deployment, and it remains independent of future interaction outcomes.

The *realized state* $\mathbf{x}_{1:F}$ is the robot state logged after the action executes in a scene. It aligns visually with the future video, but it already reflects contact, compliance, latency, failures, and other interaction outcomes. Rendering realized states as prompts provides an oracle diagnostic. The nominal trajectory is the conditioning signal available at deployment. Figure 2 shows why this intermediate signal is appropriate for conditioning the world model.

We separate the mismatch among these signals into two gaps. The *action-realization gap* is the difference between the raw action and the nominal motion produced by the controller. The *nominal-realized gap* is the difference between the nominal trajectory and the state observed during a contact-rich rollout. We assign the first gap to the robot-specific realization process. The world model must account for the second gap, along with object motion and changes in occlusion. For each robot, we instantiate Φ_R with that robot’s controller. This step resolves robot-specific action semantics before we supply the visual conditioning streams to the video model.

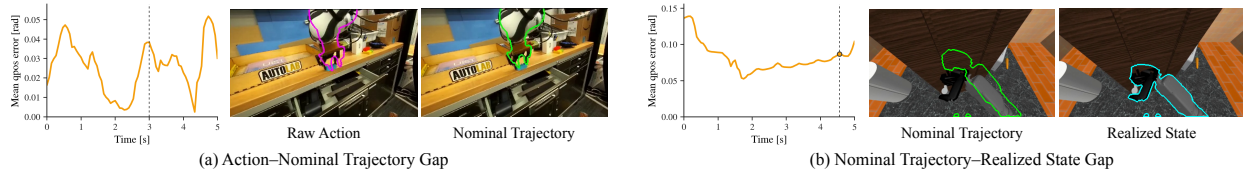


Figure 2. Action-to-state realization gaps. (a) Robot-specific controllers and hardware constraints create a gap between raw actions and nominal trajectories. (b) Scene interaction creates a gap between nominal trajectories and realized states. The nominal trajectory is the conditioning signal available at deployment.

3.3 Camera-Aware Static Context

The static context stream in eq. (3) represents the initial scene state in the target camera’s coordinates. It supplies scene appearance, scene depth, and viewpoint information independent of robot interaction. By preserving scene content in this stream, we allow the world model to focus on changes caused by interaction [44].

When we have a static scene representation and camera trajectory, we use Π_S to render static scene RGB and depth along the target video’s camera trajectory $\mathcal{C}_{1:F}$. For fixed cameras, we produce this stream by repeating the initial frame across the prediction horizon. We can therefore use the same camera-aligned conditioning interface for clips with fixed or moving viewpoints. We supply action information separately through the rendered nominal robot geometry.

3.4 Rendered Robot Action Prompts

We use the renderer Π_R in eq. (2) to render the robot’s nominal trajectory in the target camera frame. We produce RGB renderings of the robot’s URDF mesh, preserving link geometry, wrist offsets, gripper shape, and end-effector structure. The video model can thus see the action in the same visual coordinates as the target video.

RGB mesh renderings locate the robot in image space. When the end-effector overlaps an object in the image, depth distinguishes whether it is in front of the object, behind it, or at contact depth. We therefore include end-effector-only depth $\mathbf{D}_{1:F}^{\text{eef}}$ alongside static scene depth $\mathbf{D}_{1:F}^{\text{scene}}$. We focus depth rendering on the end-effector, where interaction is most likely to occur. These two depth signals help the model distinguish proximity, likely contact, and occlusion beyond what apparent 2D overlap reveals.

3.5 Learning the Scene-Response Model

We implement Eq. 4 with a latent video diffusion model. Following the residual-dynamics formulation of Dexterous World Models [44], we use a video inpainting backbone [45]. We supply the static context video as conditioning and apply a full mask over the prediction horizon. The model learns to generate scene changes caused by interaction, using the static scene context and rendered robot geometry as conditioning.

We use the pretrained video VAE [46] of the backbone model [47] to encode each visual conditioning stream. We concatenate the encodings of static RGB context, rendered robot RGB, scene depth, and end-effector depth to form c :

$$c = \left[\mathcal{E}(\mathbf{B}_{1:F}^{\text{rgb}}), \mathcal{E}(\mathbf{M}_{1:F}^{\text{rgb}}), \mathcal{E}(\mathbf{D}_{1:F}^{\text{scene}}), \mathcal{E}(\mathbf{D}_{1:F}^{\text{eef}}) \right]. \quad (5)$$

Here $\mathcal{E}$ denotes the pretrained video VAE encoder applied to each conditioning stream. We concatenate these encodings with the noisy video latent as input to the model.

We train the model by minimizing the latent flow-matching objective [48, 49]. For a sampled noise latent ϵ and noise level σ , we form $z_\sigma = (1 - \sigma)z_0 + \sigma\epsilon$. We use $u = \epsilon - z_0$ as the target velocity:

$$\mathcal{L} = \mathbb{E}_{z_0, \sigma, \epsilon} \left[\|u - v_\theta(z_\sigma, \sigma \mid c, \mathcal{T})\|_2^2 \right]. \quad (6)$$

The text prompt $\mathcal{T}$ contains scene context only; it excludes descriptions of the intended action or future outcome.

4 Experiments

4.1 Experimental Setup

Datasets. We use the enhanced-extrinsic subset of DROID [50] and a set of RoboCasa-GR1 rollouts. We collect the RoboCasa-GR1 rollouts by executing the humanoid VLA DiT4DiT [52] in RoboCasa [51] across 24 tasks, with 50 episodes per task. We process both datasets into 81-frame clips at 480×832 resolution and 16 fps, yielding 41,642 DROID clips and 9,380 RoboCasa-GR1 clips. We evaluate on 256 held-out DROID clips and 128 held-out RoboCasa-GR1 clips. We use HRDexDB [53] and DexYCB [54] for qualitative studies.

Nominalization. For DROID, we replay the raw teleoperator joint and gripper targets in a scene-free, robot-only Isaac Lab [55] environment. The robot’s own controller and actuation limits produce a physically trackable nominal trajectory. For RoboCasa-GR1, we replay the DiT4DiT [52] controller actions from the clip-start state in a collision-free shadow rollout to obtain the Fourier GR-1 nominal state. In both cases, the nominal trajectory is available at inference, and we render it to produce the robot conditioning signal.

Baselines. We compare against two baselines. *Ctrl-World* [4] is a 7-DoF Cartesian pose-conditioned world model built on Stable Video Diffusion (SVD) [56]. The *AdaLN* [57] *state vector* baseline supplies the same deployment-available nominal trajectory as numeric state values. For the SVD comparison, we retrain the same backbone with either pose conditioning or the rendered interface, then evaluate both variants on the DROID held-out set. For Wan [47]/Wan2.1-Fun InP [45], we compare numeric state-vector conditioning with our rendered mesh-and-depth interface.

4.2 Main Comparison

Quantitative comparison. We first ask whether the rendered interface represents actions more effectively for the model than vector- or pose-conditioned alternatives. We compute PSNR, SSIM, and LPIPS [58] for each clip and report their averages over the evaluation set. The rendered interface outperforms the Ctrl-World-style Cartesian pose condition in the SVD comparison, which holds the backbone fixed. It also outperforms the AdaLN state-vector baseline on both DROID and RoboCasa-GR1 in the jointly trained Wan setting. These results support our central interface design. Rendered robot geometry gives the video model direct pixel-space evidence of robot motion. Numeric action or state conditions instead require the model to learn an additional map from those values to visible motion. We compare results only within each backbone because the backbone groups differ in resolution and training data.

Qualitative comparison. Figure 3 emphasizes cases in which reconstruction metrics are least diagnostic: small object motion, brief contact, occlusion ordering, missed grasps, and no-contact outcomes. The AdaLN state-vector baseline often preserves scene appearance but depicts the intended action weakly or in

Table 1. Main action-interface comparison. Each backbone cell lists the model, its training resolution, and its training data (D+R: DROID+RoboCasa-GR1). SVD 1.5B trains on DROID only, so we do not evaluate it on RoboCasa-GR1 (dashes). Lower resolution inflates reconstruction metrics, so numbers are comparable only within a backbone. We report DROID and RoboCasa-GR1 separately rather than averaging them; **bold** marks the best result within each backbone.

Backbone	Method	DROID			RoboCasa-GR1		
		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
SVD 1.5B 192 $\times$ 320, DROID	Ctrl-World pose [4]	23.15	0.884	0.104	–	–	–
	Rendered mesh (ours)	25.05	0.899	0.091	–	–	–
Wan 2.1 14B 480 $\times$ 832, D+R	AdaLN state vector	18.57	0.824	0.224	17.67	0.835	0.194
	Rendered mesh + EEF/scene depth (ours)	21.87	0.859	0.178	24.61	0.889	0.131

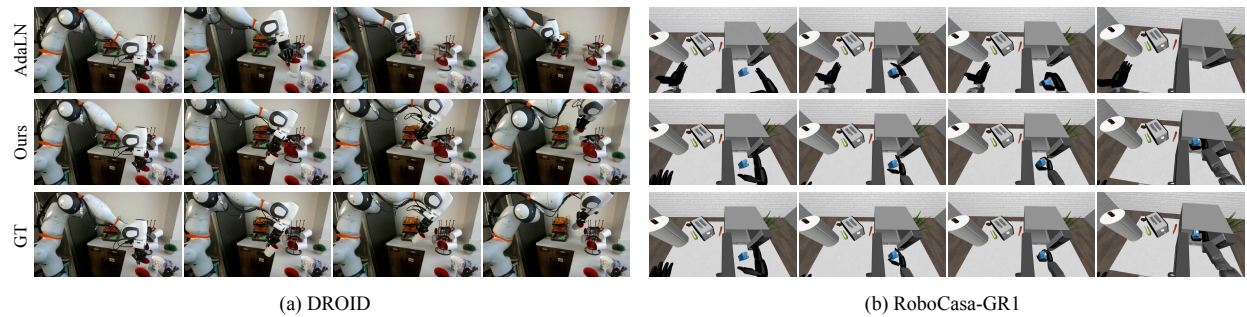


Figure 3. Qualitative comparison. Rendered robot geometry localizes robot-driven scene changes, while depth helps resolve contact-relevant proximity and occlusion.

the wrong location. Rendered robot geometry makes the action visible directly in the target camera frame. The resulting predictions localize scene changes more accurately around the robot motion and contact region.

4.3 Action Realization and Depth Ablations

Table 2 isolates the two factors introduced in Sections 3.2 and 3.4: nominal trajectory conditioning and the depth-aware rendered interface. We train and evaluate all variants on DROID, keeping the dataset and backbone fixed.

Table 2. Ablation study on DROID.

Variant	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Raw action mesh	21.57	0.860	0.175
Nominal mesh	22.44	0.872	0.164
Nominal mesh + EEF/scene depth	23.08	0.874	0.161

Nominal trajectory conditioning. The first two rows compare raw-action mesh rendering with nominal trajectory rendering. Both variants use only mesh RGB for conditioning. This comparison isolates the action-realization gap. The raw-action mesh shows the command before the robot-local stack realizes it,

whereas the nominal trajectory rendering shows the controller-realized robot motion available at deployment. Nominal trajectory rendering improves over the raw-action mesh across metrics (Table 2). This result shows that assigning action realization to the robot-local stack produces better aligned robot renderings from motion available at deployment.

Depth-aware rendered interface. The final row adds end-effector and scene depth to nominal trajectory rendering, testing the depth-aware interface from Section 3.4. RGB mesh rendering locates robot motion in the image plane. The two depth streams provide 3D proximity and occlusion-order information relevant to contact. Adding depth improves all three metrics further (Table 2). Figure 4 illustrates the qualitative effect. Without depth, the model can treat image-plane overlap as contact even when the object is farther away.

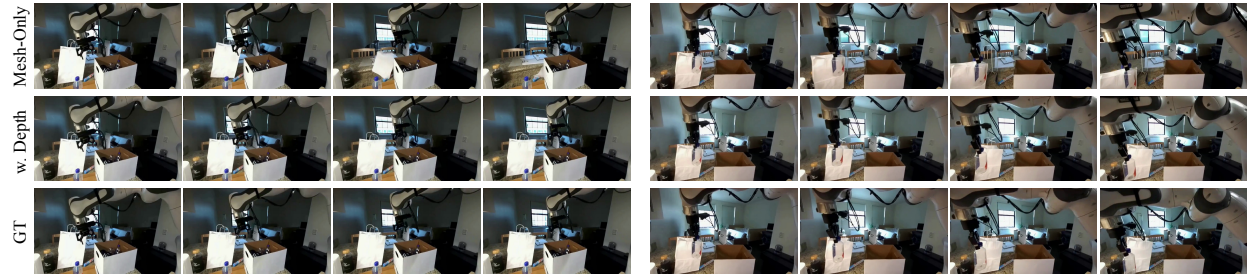


Figure 4. Depth ablation. End-effector and scene depth help distinguish contact-relevant proximity from image-plane overlap.

4.4 Qualitative Interface Analysis

We study two qualitative properties of the rendered interface: prompt following and embodiment generalization. We probe prompt following by changing the rendered robot motion (Figure 5) and embodiment generalization by changing the robot geometry (Figure 6).

Counterfactual trajectory edits. On real DROID episodes, we keep the initial scene fixed and edit the robot trajectory. We re-solve the edited trajectory with cuRobo [59] under the same Panda kinematics, then compare its rendering with the original nominal robot rendering. Figure 5 shows that changing only the rendered nominal trajectory changes the predicted scene response. This indicates that the model uses the rendered robot geometry as its action signal.

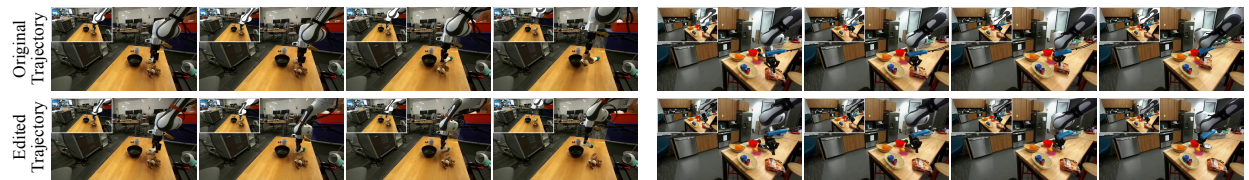


Figure 5. Prompt-following probe. Changing only the rendered nominal trajectory changes the predicted scene response.

Unseen embodiment composition. Figure 6 uses HRDexDB [53], which provides an unseen pairing of an xArm 6 arm and an Inspire F1 hand. We render the held-out robot motion as camera-aligned URDF geometry and pass it through the same trained model. The conditioning mechanism remains fixed, and only the rendered robot geometry changes. This demonstrates embodiment generalization at the interface level. The same visual conditioning path can accept unseen robot geometry once we represent it as a rendering.

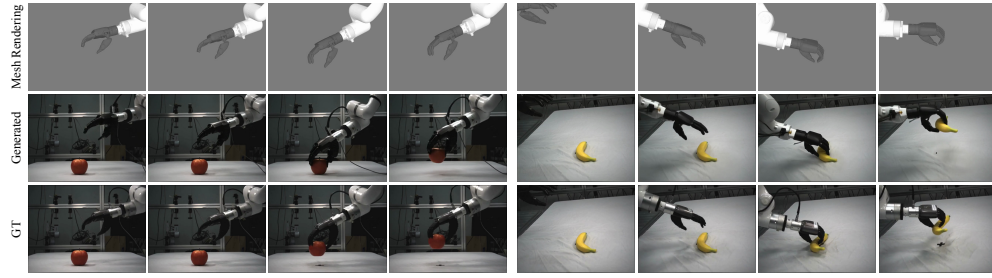


Figure 6. Zero-shot embodiment composition. HRDexDB contains an unseen xArm 6–Inspire F1 pairing; rendered URDF motion still drives the predicted scene response.

4.5 Application: Human Demonstration to Robot Video

Human demonstration to robot video. We apply the shared rendered interface to human manipulation videos from DexYCB [54] (Figure 7). We retarget the hand motion to a robot [60, 61] and render the result through the same mesh-and-depth interface used for robot data. The motion in this application originates from retargeted human demonstrations. It therefore illustrates that the rendered interface is independent of controller-specific action representations: the world model receives rendered robot geometry regardless of the motion source.

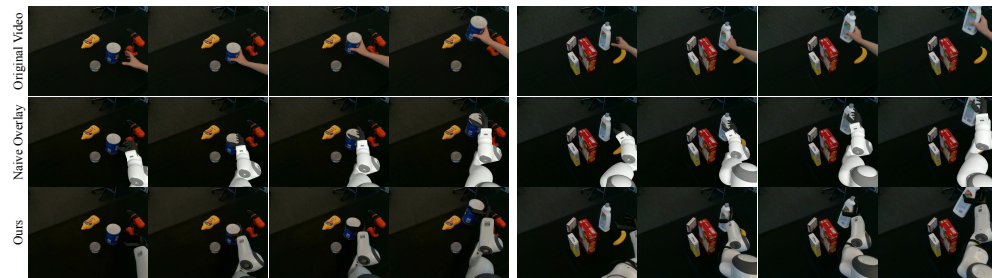


Figure 7. Application: human demonstration to robot video. Retargeted human motion is rendered as robot geometry and converted into a robot interaction rollout.

5 Conclusion and Limitations

We present robot-factored world models, which handle action realization and robot rendering outside the video world model. The robot-local stack converts actions into nominal trajectories that are available at deployment. We render these trajectories as camera-aligned robot geometry, so the model receives visible robot motion and learns how the surrounding scene responds. Experiments on DROID and RoboCasa-GR1 show that the rendered interface outperforms vector-conditioned baselines, and the same interface extends to unseen robot embodiments and retargeted human demonstrations.

Our rendering-based action interface improves action following and generalization, but several limitations remain. The interface requires a known robot URDF and camera-to-robot calibration for each new embodiment. These are standard in calibrated setups. When the camera moves, the static context stream assumes a static scene representation. Simulation provides the full representation, whereas real-world observations cover only part of the scene. Feedforward 3D reconstruction can supply the representation, though closing the appearance gap may require additional training. Finally, DROID episodes are predominantly successful, so the model sees few grasp failures, slips, and mispredicted contacts. Collecting failure data would improve robustness in these cases.

References

- [1] D. Ha and J. Schmidhuber. Recurrent world models facilitate policy evolution. In *NeurIPS*, 2018.
- [2] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun. Navigation world models. In *CVPR*, 2025.
- [3] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong. Irasim: A fine-grained world model for robot manipulation. In *ICCV*, 2025.
- [4] Y. Guo, L. X. Shi, J. Chen, and C. Finn. Ctrl-world: A controllable generative world model for robot manipulation. In *ICLR*, 2026.
- [5] S. Gao, W. Liang, K. Zheng, A. Malik, S. Ye, S. Yu, W.-C. Tseng, Y. Dong, K. Mo, C.-H. Lin, Q. Ma, S. Nah, L. Magne, J. Xiang, Y. Xie, R. Zheng, D. Niu, Y. L. Tan, K. Zentner, G. Kurian, S. Indupuru, P. Jannaty, J. Gu, J. Zhang, J. Malik, P. Abbeel, M.-Y. Liu, Y. Zhu, J. Jang, and L. Fan. Dreamdojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.
- [6] A. K. Sharma, Y. Sun, N. Lu, Y. Zhang, J. Liu, and S. Yang. World-gymnast: Training robots with reinforcement learning in a world model. *arXiv preprint arXiv:2602.02454*, 2026.
- [7] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang. Worldgym: World model as an environment for policy evaluation. *arXiv preprint arXiv:2506.00613*, 2025.
- [8] Y. Wang, C. Wen, H. Guo, S. Peng, M. Qin, H. Bao, X. Zhou, and R. Hu. Precise action-to-video generation through visual action prompts. In *ICCV*, 2025.
- [9] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.
- [10] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [11] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg. Daydreamer: World models for physical robot learning. In *CoRL*, 2023.
- [12] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. In *ICLR*, 2024.
- [13] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.
- [14] J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, Y. Fang, F. Hu, S. Huang, K. Kundalia, Y.-C. Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025.
- [15] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *ICML*, 2024.
- [16] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in atari. *NeurIPS*, 2024.
- [17] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter. Diffusion models are real-time game engines. In *ICLR*, 2025.
- [18] Z. Yang, Y. Chen, J. Wang, S. Manivasagam, W.-C. Ma, A. J. Yang, and R. Urtasun. Unisim: A neural closed-loop sensor simulator. In *CVPR*, 2023.
- [19] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long. ivideogpt: Interactive videogpts are scalable world models. *NeurIPS*, 2024.
- [20] H. Zhu, Y. Wang, J. Zhou, W. Chang, Y. Zhou, Z. Li, J. Chen, C. Shen, J. Pang, and T. He. Aether: Geometric-aware unified world modeling. In *ICCV*, 2025.
- [21] J. Zhou, H. Gao, V. Voleti, A. Vasishta, C.-H. Yao, M. Boss, P. Torr, C. Rupprecht, and V. Jampani. Stable virtual camera: Generative view synthesis with diffusion models. In *ICCV*, 2025.
- [22] L. Russell, A. Hu, L. Bertoni, G. Fedoseev, J. Shotton, E. Arani, and G. Corrado. Gaia-2: A controllable multi-view generative world model for autonomous driving. *arXiv preprint arXiv:2503.20523*, 2025.
- [23] S. Gao, S. Zhou, Y. Du, J. Zhang, and C. Gan. Adaworld: Learning adaptable world models with latent actions. *arXiv preprint arXiv:2503.18938*, 2025.
- [24] M. Huang, Y. Xiang, Z. Liang, J. Huang, J. Wang, Z. Xu, F. Tan, H. Zhou, M. Yang, and G. Che. Coworld-vla: Thinking in a multi-expert world model for autonomous driving. *arXiv preprint arXiv:2605.10426*, 2026.

- [25] A. Liang, P. Czempin, M. Hong, Y. Zhou, E. Biyik, and S. Tu. Clam: Continuous latent action models for robot learning from unlabeled demonstrations. *arXiv preprint arXiv:2505.04999*, 2025.
- [26] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [27] Q. Garrido, T. Nagarajan, B. Terver, N. Ballas, Y. LeCun, and M. Rabbat. Learning latent action world models in the wild. *arXiv preprint arXiv:2601.05230*, 2026.
- [28] L. Zhang, A. Rao, and M. Agrawala. Adding conditional control to text-to-image diffusion models. In *ICCV*, 2023.
- [29] Y. Zhang, Y. Wei, X. ZHANG, W. Zuo, Q. Tian, et al. Controlvideo: Training-free controllable text-to-video generation. In *ICLR*, 2024.
- [30] X. Wang, H. Yuan, S. Zhang, D. Chen, J. Wang, Y. Zhang, Y. Shen, D. Zhao, and J. Zhou. Videocomposer: Compositional video synthesis with motion controllability. *NeurIPS*, 2023.
- [31] Y. Guo, C. Yang, A. Rao, Z. Liang, Y. Wang, Y. Qiao, M. Agrawala, D. Lin, and B. Dai. Animatediff: Animate your personalized text-to-image diffusion models without specific tuning. *arXiv preprint arXiv:2307.04725*, 2023.
- [32] Z. Wang, Z. Yuan, X. Wang, Y. Li, T. Chen, M. Xia, P. Luo, and Y. Shan. Motionctrl: A unified and flexible motion controller for video generation. In *ACM SIGGRAPH*, 2024.
- [33] S. Yin, C. Wu, J. Liang, J. Shi, H. Li, G. Ming, and N. Duan. Dragnuwa: Fine-grained control in video generation by integrating text, image, and trajectory. *arXiv preprint arXiv:2308.08089*, 2023.
- [34] Z. Zhang, J. Liao, M. Li, Z. Dai, B. Qiu, S. Zhu, L. Qin, and W. Wang. Tora: Trajectory-oriented diffusion transformer for video generation. In *CVPR*, 2025.
- [35] M. Niu, X. Cun, X. Wang, Y. Zhang, Y. Shan, and Y. Zheng. Mofa-video: Controllable image animation via generative motion field adaptations in frozen image-to-video diffusion model. In *ECCV*, 2024.
- [36] H. Zhou, C. Wang, R. Nie, J. Liu, D. Yu, Q. Yu, and C. Wang. Trackgo: A flexible and efficient method for controllable video generation. In *AAAI*, 2025.
- [37] Z. Jiang, Z. Han, C. Mao, J. Zhang, Y. Pan, and Y. Liu. Vace: All-in-one video creation and editing. In *ICCV*, 2025.
- [38] D. Geng, C. Herrmann, J. Hur, F. Cole, S. Zhang, T. Pfaff, T. Lopez-Guevara, Y. Aytar, M. Rubinstein, C. Sun, et al. Motion prompting: Controlling video generation with motion trajectories. In *CVPR*, 2025.
- [39] J. Shin, Z. Li, R. Zhang, J.-Y. Zhu, J. Park, E. Shechtman, and X. Huang. Motionstream: Real-time video generation with interactive motion controls. *arXiv preprint arXiv:2511.01266*, 2025.
- [40] H. Qiu, Z. Chen, Z. Wang, Y. He, M. Xia, and Z. Liu. Freetraj: Tuning-free trajectory control in video diffusion models. *arXiv preprint arXiv:2406.16863*, 2024.
- [41] W.-D. K. Ma, J. P. Lewis, and W. B. Kleijn. Trailblazer: Trajectory control for diffusion-based video generation. In *ACM SIGGRAPH Asia*, 2024.
- [42] Y. Jain, A. Nasery, V. Vineet, and H. Behl. Peekaboo: Interactive video generation via masked-diffusion. In *CVPR*, 2024.
- [43] Y. Li, X. Wang, Z. Zhang, Z. Wang, Z. Yuan, L. Xie, Y. Shan, and Y. Zou. Image conductor: Precision control for interactive video synthesis. In *AAAI*, 2025.
- [44] B. Kim, T. Kim, J. Lee, and H. Joo. Dexterous world models. *arXiv preprint arXiv:2512.17907*, 2025.
- [45] aigc apps. VideoX-fun: A video generation pipeline for diffusion transformer, 2026. URL <https://github.com/aigc-apps/VideoX-Fun>.
- [46] D. P. Kingma and M. Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
- [47] Team Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [48] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *ICLR*, 2023.
- [49] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *ICLR*, 2023.
- [50] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. In *RSS*, 2024.

- [51] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *RSS*, 2024.
- [52] T. Ma, J. Zheng, Z. Wang, C. Jiang, A. Cui, J. Liang, and S. Yang. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv preprint arXiv:2603.10448*, 2026.
- [53] J. Lim, T. Ha, M. Choi, J. Kim, B. Kim, S. Jeon, and H. Joo. Hrdexdb: A large-scale dataset of dexterous human and robotic hand grasps. *arXiv preprint arXiv:2604.14944*, 2026.
- [54] Y.-W. Chao, W. Yang, Y. Xiang, P. Molchanov, A. Handa, J. Tremblay, Y. S. Narang, K. Van Wyk, U. Iqbal, S. Birchfield, J. Kautz, and D. Fox. DexYCB: A benchmark for capturing hand grasping of objects. In *CVPR*, 2021.
- [55] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Muñoz, X. Yao, R. Zürbrugg, N. Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.
- [56] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.
- [57] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *ICCV*, 2023.
- [58] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [59] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, N. Ratliff, and D. Fox. curobo: Parallelized collision-free minimum-jerk robot motion generation. *arXiv preprint arXiv:2310.17274*, 2023.
- [60] Y. Qin, W. Yang, B. Huang, K. Van Wyk, H. Su, X. Wang, Y.-W. Chao, and D. Fox. Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system. In *RSS*, 2023.
- [61] C. M. Kim, B. Yi, H. Choi, Y. Ma, K. Goldberg, and A. Kanazawa. Pyroki: A modular toolkit for robot kinematic optimization. In *IROS*, 2025.
- [62] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [63] B. Wen, M. Trepte, J. Aribido, J. Kautz, O. Gallo, and S. Birchfield. Foundationstereo: Zero-shot stereo matching. In *CVPR*, 2025.
- [64] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [65] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [66] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [67] L. Contributors. Lightx2v: Light video generation inference framework, 2025. URL <https://github.com/ModelTC/lightx2v>.
- [68] B. Zi, W. Peng, X. Qi, J. Wang, S. Zhao, R. Xiao, and K.-F. Wong. Minimax-remover: Taming bad noise helps video object removal. *Advances in Neural Information Processing Systems*, 38:75518–75547, 2026.
- [69] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu. DexMimicGen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *ICRA*, pages 16923–16930, 2025.

A Interface Construction Details

Dataset-specific nominalization. The main paper distinguishes raw actions, nominal robot-only trajectories, and realized robot states. DROID [50] contains demonstrations collected through human teleoperation. It provides standardized robot action representations, including joint-space and end-effector-space actions. Our preprocessing uses the logged joint and gripper target sequence as the raw action for nominal replay. These targets can change faster than the arm can physically track them. Rendering the targets directly therefore often produces motion that runs ahead of the recorded video. We replay the action in a scene-free, robot-only Isaac Lab [55] environment. The Panda controller and actuation limits shape the raw action sequence into a physically trackable nominal trajectory.

For RoboCasa-GR1, the DiT4DiT [52] policy outputs 29D controller actions, while the renderer consumes the 39D Fourier GR-1 joint/hand/waist state. We replay each action sequence from the clip-start state in a collision-free shadow rollout and render the resulting Fourier GR-1 nominal state. The RoboCasa-GR1 corpus contains episode trajectories generated by executing a DiT4DiT VLA policy in the Fourier GR-1 humanoid tabletop setup. The GR00T [62] project released this setup on top of the RoboCasa [51] simulator.¹ In both robot datasets, nominal robot geometry serves as the world model’s deployment-compatible conditioning signal.

Static context and scene descriptions. The static context stream follows the camera trajectory of each target clip while removing robot interaction. For fixed-view DROID clips, we repeat the initial RGB observation across the prediction horizon to form the context. When we enable scene depth, we estimate metric depth from the ZED stereo observations with FoundationStereo [63] and align it to the same fixed camera frame. For RoboCasa-GR1 dynamic-view clips, the simulator provides both the camera trajectory and the robot-free scene. We render static RGB and static depth along that trajectory without the robot. These streams provide scene context that follows the camera, while the nominal robot mesh and end-effector depth streams carry action information.

We pair each clip with a scene description $\mathcal{T}$ generated by Qwen3-VL [64]. This language condition describes only the visible scene appearance and context. It supplies scene-level semantic context, while the nominal robot geometry carries action information.

B Model, Baseline, and Evaluation Details

Wan adaptation. We build the Wan [47] 2.1 14B variants on the pretrained Wan2.1-Fun-V1.1-14B-InP image-to-video model from VideoX-Fun [45]. We make two trainable additions and freeze all other pretrained weights: (i) we extend the patch-embedding projection to accept the extra conditioning channels, and (ii) we attach LoRA [65] weights with rank 64 and $\alpha=64$ to the transformer. The base model has 36 input latent channels. We encode each rendered stream with the VAE to obtain a 16-channel latent, then concatenate these latents along the channel axis. The patch embedding therefore grows to $36+C$ input channels, where C is the number of conditioning channels in the variant. We train the extended patch-embedding weights together with the LoRA weights. Training uses the latent flow-matching objective [48, 49]. We form noisy latents by linearly interpolating between data and noise, and the DiT predicts the corresponding velocity.

¹<https://github.com/robocasa/robocasa-gr1-tabletop-tasks>

Conditioning channels. Each rendered stream contributes 16 latent channels:

- *Mesh RGB only* (raw action mesh or nominal trajectory rendering): $C = 16$.
- *Mesh RGB + end-effector depth + scene depth*: $C = 48$.

The Wan2.1-Fun InP image-conditioning path carries the static RGB context. The extra rendered-stream path carries the robot mesh and depth conditions. The static video contributes 16 latent channels, and the inpainting mask contributes four time-compressed mask channels. We set the mask to all-known pixels ($m=1$ in our convention). The static stream therefore preserves scene appearance, while the rendered robot streams explain action-conditioned changes. This separation of static scene conditioning from action-conditioned interaction dynamics is inspired by Dexterous World Models [44].

AdaLN state-vector baseline. We use AdaLN [57] conditioning to inject numeric state vectors into the diffusion transformer. For DROID, the AdaLN baseline uses a per-frame 7D nominal end-effector state: $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}, g]$. We compute this state using Panda forward kinematics from the robot-only Isaac Lab nominal joint rollout and the nominal gripper scalar g . For RoboCasa-GR1, the DiT4DiT [52] policy action is a 29D controller action, ordered as left arm 7, right arm 7, left hand 6, right hand 6, and waist 3. After replay from the clip-start state, the AdaLN baseline uses the resulting 39D nominal Fourier GR-1 state, ordered as left arm 7, right arm 7, left hand 11, right hand 11, and waist 3. For joint DROID+RoboCasa training, we use a zero-padded 46D union vector. DROID fills its first 7 entries, and RoboCasa fills the next 39 entries. We stack four raw-frame states for each Wan latent frame, giving a 184D AdaLN input.

Table 3. Training hyperparameters for the Wan 2.1 14B world model.

Hyperparameter	Value
Base model	Wan2.1-Fun-V1.1-14B-InP
LoRA rank / α	64 / 64
Trainable parameters	extended patch embedding + LoRA
Optimizer	AdamW [66] ($\beta_1=0.9, \beta_2=0.95$)
Weight decay	0.01
Learning rate	1×10^{-4}
LR schedule	cosine with restarts, 100 warmup steps
Gradient clipping	1.0
Batch size / grad. accum.	1 / 4 per GPU
Precision	bf16
Gradient checkpointing	enabled
Resolution	480×832
Clip length / fps	81 frames / 16

Inference. We sample the Wan 2.1 14B variants with the LightX2V [67] CFG-and-step-distilled LoRA at 4 denoising steps and guidance scale 1.0. SVD-based variants use 25 Euler steps at 192×320 . The LightX2V distillation LoRA applies directly because our adaptations to the pretrained Wan 2.1 14B transformer consist of a patch-embedding projection and task-specific LoRA weights. It reduces the number of denoising steps by $10\times$.

C Oracle-State Diagnostic

Offline datasets contain logged realized robot states, so one could render these states as conditioning prompts to train the video diffusion model. However, these states become observable only after the action interacts with the scene. They therefore serve as an oracle diagnostic. Table 4 examines three settings, all of which condition on rendered robot mesh RGB without depth. Nominal/nominal is our deployment-compatible design. It renders both training and inference prompts from nominal robot states computed before scene interaction. Logged/logged uses realized future states at both training and inference, making it an oracle diagnostic. Logged/nominal trains the video model with prompts rendered from logged realized states. At inference, it uses prompts rendered from nominal robot states to satisfy the deployment-time availability constraint. Nominal/nominal performs best among the deployment-compatible settings. This result shows the value of using nominal trajectory prompts, which are available before scene interaction, for both training and inference.

Table 4. Oracle-state conditioning diagnostic. Logged/logged uses future realized robot states as an oracle diagnostic. Logged/nominal tests whether training prompts from logged states are compatible with nominal prompts available at deployment.

Dataset	Train state	Inference state	PSNR	SSIM	LPIPS
DROID	nominal	nominal	21.74	0.856	0.179
DROID	logged	logged	22.37	0.861	0.174
DROID	logged	nominal	21.12	0.846	0.187
RoboCasa-GR1	nominal	nominal	25.70	0.904	0.120
RoboCasa-GR1	logged	logged	28.26	0.918	0.112
RoboCasa-GR1	logged	nominal	24.69	0.897	0.126

D Human Demonstration to Robot Video Pipeline

Figure 8 illustrates how we convert a DexYCB [54] RGB-D clip into the same rendered robot interface used for robot demonstrations. The clip provides 21-point 3D hand keypoints for each frame.

Scene inpainting. When the initial frame contains a visible human hand, we optionally remove it with a video inpainting model [68]. This produces a robot-free scene image for the static RGB context. Clips without a visible hand in the initial frame use the original scene observation.

Hand retargeting. We use the AnyTeleop [60] retargeting setup to map human fingertip motion to the robot hand.

Arm retargeting. The wrist trajectory and hand orientation define the target end-effector motion. We map this motion to the robot arm with PyRoki [61] inverse kinematics.

Mesh rendering and conditioning assembly. We render the combined Panda arm and Inspire hand trajectory in the DexYCB camera frame using the calibrated robot geometry. We then assemble the same conditioning streams used by the robot world model: static RGB, static depth, robot mesh video, and end-effector depth. The same trained world model receives these streams and generates a robot manipulation rollout from the human-sourced motion.

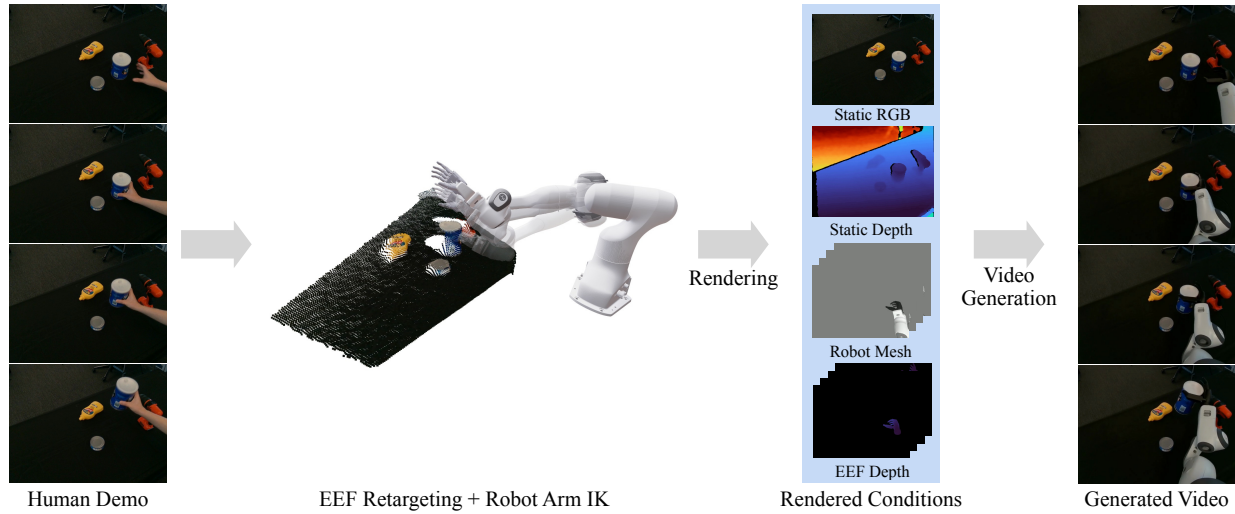


Figure 8. Human demonstration to robot video pipeline. We convert human motion into the same rendered mesh-and-depth interface used for robot data, then pass the rendered streams to the world model.

E Additional Qualitative Results

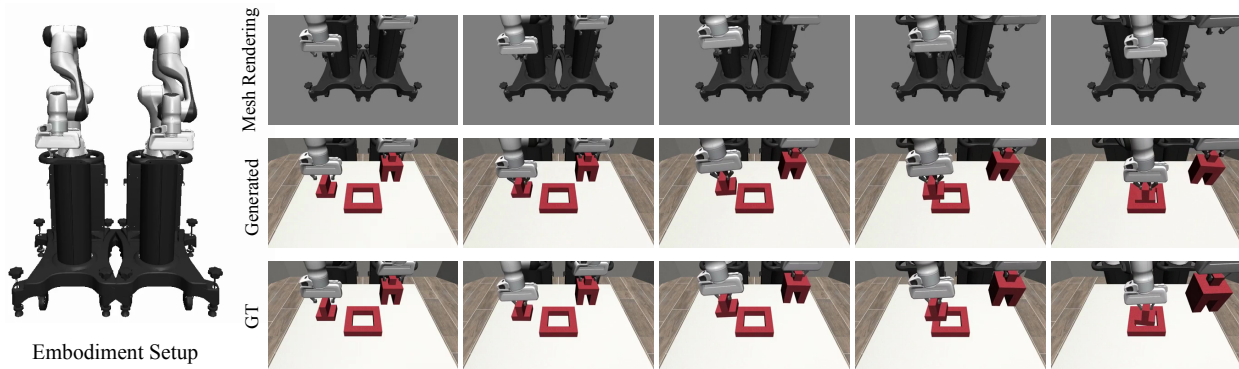


Figure 9. Zero-shot multi-embodiment inference. We render a bimanual Franka Panda setup from DexMimicGen as robot geometry and pass the rendering to the same video model.

Zero-shot multi-embodiment inference. Figure 9 probes zero-shot inference on a DexMimicGen [69] scenario with two Franka Panda arms and parallel-jaw grippers. The new multi-arm prompt represents the robot as rendered geometry in the camera frame. The same video model can therefore accept it through the rendered interface. This qualitative result illustrates how the rendered interface supports new embodiment and end-effector configurations through the same conditioning path.

DROID and RoboCasa-GR1 comparisons. Figure 10 provides additional qualitative comparisons, with DROID examples on the left and RoboCasa-GR1 examples on the right. Across these examples, AdaLN state-vector conditioning can preserve scene appearance but often depicts the intended robot motion weakly or in the wrong location. Rendered robot geometry makes the action visible in the target camera frame. The resulting videos follow the commanded motion more consistently and localize scene changes around the robot and contact region.

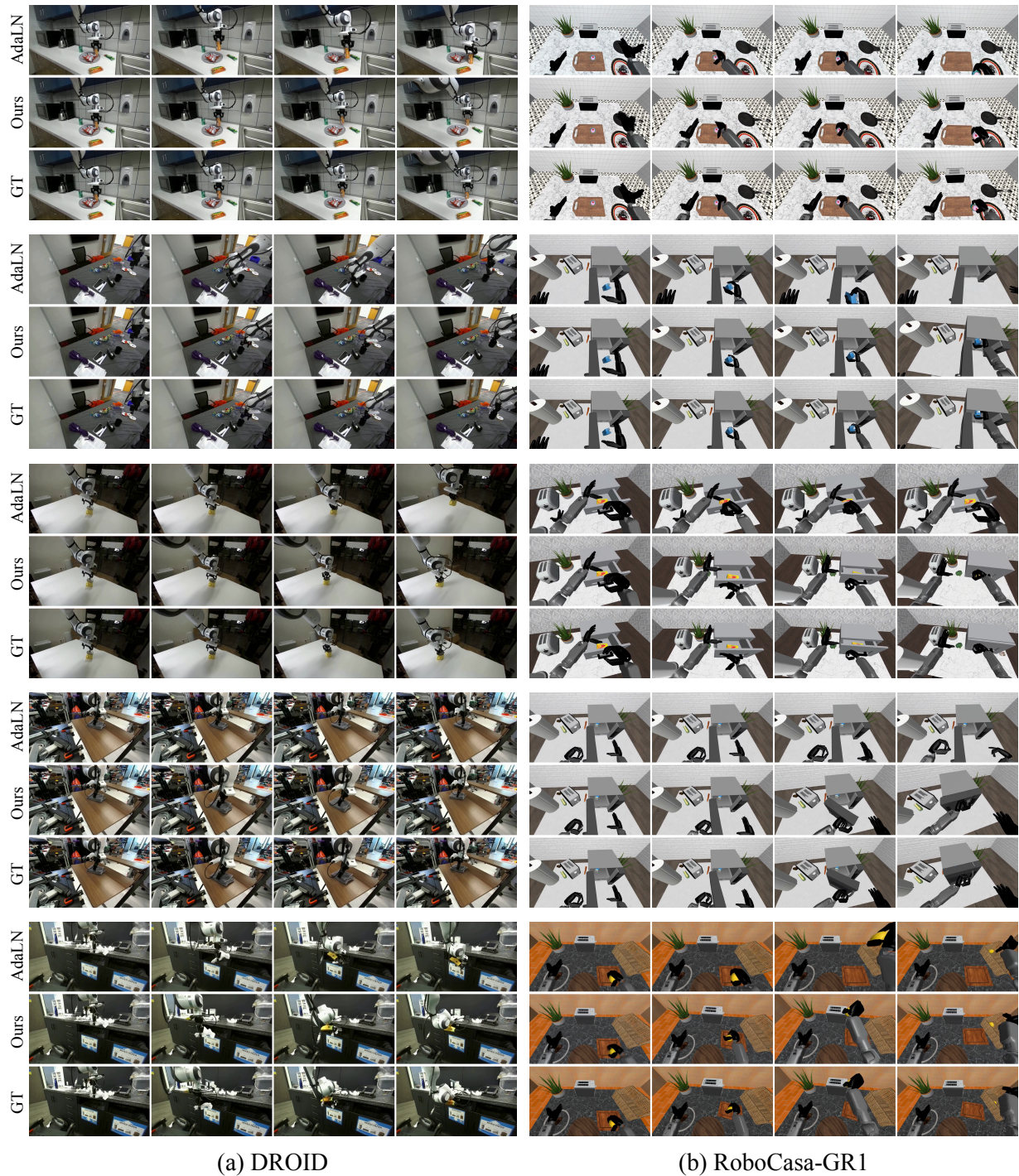


Figure 10. Additional qualitative comparisons on DROID and RoboCasa-GR1. Rendered robot geometry makes the action visible in the target camera frame. Videos generated with this conditioning follow the commanded robot motion more consistently than those generated with AdaLN state-vector conditioning.